

OOP in C++

La programmazione orientata agli oggetti (OOP) in C++ consente di sviluppare codice pulito, riutilizzabile e scalabile organizzando i programmi attorno a oggetti e classi. Imparerai a definire e utilizzare classi, creare costruttori e distruttori, e applicare l'incapsulamento per proteggere i dati. Argomenti come ereditarietà, polimorfismo, membri statici, controllo degli accessi e overload degli operatori ti aiuteranno a progettare strutture di codice flessibili ed efficienti.

Fondamenti della OOP in C++

Introduzione alla Programmazione Orientata agli Oggetti in C++



Prerequisiti

- ✓ Introduzione a C++
- ✓ Puntatori e Riferimenti in C++
- ✓ Funzioni in C++



Definition

Programmazione Orientata agli Oggetti è un paradigma che organizza la progettazione del software attorno al concetto di classi e oggetti, che possono contenere dati. L'idea chiave della OOP è modellare entità del mondo reale e le loro interazioni all'interno di un programma.

Vantaggi dell'utilizzo della OOP nello sviluppo software

La **OOP** è popolare nello sviluppo software moderno perché offre un approccio **modulare** e **strutturato** alla scrittura del codice. Favorisce la **riutilizzabilità**, la **manutenibilità** e la **scalabilità** del codice, consentendo agli sviluppatori di creare sistemi **estensibili**, **facili da comprendere** e **semplici da modificare**.

Concetto	Descrizione
Organizzazione del codice migliorata	Struttura modulare per una migliore leggibilità.
Scalabilità del codice	Riutilizza il codice in nuove classi per maggiore efficienza.
Flessibilità e adattabilità	Adatta e modifica facilmente il codice per esigenze in evoluzione.
Incapsulamento	Raggruppa in modo sicuro dati e metodi nel codice.

Perché Utilizzare la Programmazione Orientata agli Oggetti?

Immagina di dover sviluppare un'applicazione software per gestire i dati degli studenti. All'inizio potrebbe sembrare semplice. Potresti iniziare utilizzando semplici variabili per memorizzare il nome e il GPA di uno studente.

```
int main()
{
    std::string student_name = "Bob";
    float grade_point_average = 3.3;
}
```

Questo approccio funziona bene per gestire un singolo studente, ma la situazione si complica con più studenti. Una soluzione ingenua potrebbe prevedere variabili separate per ciascuno, ma questo diventa rapidamente ingestibile. Utilizzare array è un'opzione migliore per ridurre la ripetizione e migliorare l'organizzazione.

```
int main()
{
    std::string names[2] { "Bob", "Ann" };
    float avarage_grades[2] { 3.3, 3.7 };
}
```

Questo metodo funziona per ora. Ma cosa succede se è necessario memorizzare più dettagli, come **numeri di telefono**, **date di iscrizione** o **elenchi di corsi**? Con l'aumentare dei dati, gestire molteplici **array** separati diventa **complesso** e soggetto a **errori**.

```
int main()
{
    std::string names[2] { "Bob", "Ann" };
    float average_grades[2] { 3.3f, 3.7f };
    std::string emails[2] { "bob@example.com", "ann@example.com" };
}
```

Per gestire questi dati, potresti creare **funzioni** che accettano questi valori come **argomenti** per eseguire azioni come stampare le informazioni dello studente, aggiornare il **GPA** o modificare l'**indirizzo email**. Tuttavia, con più studenti e campi, finisci per ripetere gli stessi argomenti in molte funzioni, rendendo il codice **soggetto a errori** e difficile da **mantenere**.

```
void get_student_info(int id, std::string names[], float gpa[],
std::string emails[]);
void set_student_info(int id, std::string names[], float gpa[],
std::string emails[]);

void set_student_name(int id, std::string names[], float gpa[],
std::string emails[]);
void set_student_gpa(int id, std::string names[], float gpa[],
std::string emails[]);
void set_student_email(int id, std::string names[], float gpa[],
std::string emails[]);
```

Anche compiti semplici richiedono di passare ripetutamente lo stesso insieme di **array**. Aggiungendo altri campi, il codice diventa sempre più **complesso** e **ripetitivo**.

La **Programmazione Orientata agli Oggetti (OOP)** risolve questo problema permettendo di raggruppare **dati** e **comportamenti** correlati in un unico contenitore chiamato **classe**. Questo non solo **semplifica il codice**, ma migliora anche l'**incapsulamento** controllando l'accesso ai dettagli interni tramite **interfacce chiare e ben definite**.



Note

Anche se puoi utilizzare `struct` per raggiungere obiettivi simili, non può sostituire completamente **OOP** perché manca di funzionalità essenziali necessarie per creare applicazioni scalabili e manutenibili.

Sintassi di Creazione delle Classi

I concetti fondamentali della Programmazione Orientata agli Oggetti (OOP) sono **classi** e **oggetti**. Tutto nella OOP è costruito attorno a questi due elementi, che costituiscono la base per scrivere codice modulare e riutilizzabile.

- **Classe:** rappresenta un modello o uno schema per la creazione di **oggetti**. Definisce un insieme di **attributi e metodi** che saranno comuni a tutte le istanze di quella classe;
- **Oggetto:** è un'istanza di una `class`. È una realizzazione concreta di essa, con valori specifici per i suoi attributi.

Sintassi della Classe

La creazione di una classe è un processo semplice. Una classe viene definita utilizzando la parola chiave `class` seguita dal nome della `class` e da una coppia di parentesi graffe `{ }`. Per creare un **oggetto** si utilizza il nome della `class` come **tipo di dato per la variabile**.

```
#include <iostream>
#include <typeinfo>

class Student {

};

int main()
{
    Student student;
    std::cout << "Type of student: " << typeid(student).name();
}
```

Attributi della Classe



Definition

Attributi, chiamati anche proprietà o campi, sono variabili dichiarate all'interno di una classe che definiscono lo stato o le caratteristiche dei suoi oggetti.

```
class Name {  
public:  
    Type AttributeName;  
};
```

Gli attributi e i metodi pubblici in una classe possono essere **acceduti direttamente** utilizzando l'operatore punto (`.`) su un oggetto della classe.

```
class Student {  
public:  
    int age;  
};  
  
int main()  
{  
    Student bob;  
    Student ann;  
  
    bob.age = 25;  
    ann.age = 33;  
}
```



Note

Non preoccuparti del `public` keyword. Verrà spiegata più avanti.

Metodi della Classe

I metodi in una **classe** sono essenzialmente **funzioni definite all'interno della classe**. Vengono utilizzati per definire i comportamenti o le azioni che gli **oggetti** della **classe** possono eseguire.



Note

Tipici **metodi** che una `class Car` potrebbe includere:

- `accelerate()`
- `brake()`
- `honk()`

I metodi spesso manipolano gli attributi della **classe** o eseguono operazioni rilevanti per gli **oggetti**.

Implementazione dei metodi al di fuori della classe

I metodi possono essere definiti al di fuori della dichiarazione della `class` utilizzando l'**operatore di risoluzione dell'ambito** (`::`). Questo viene spesso fatto per separare la dichiarazione nel file header dalla sua implementazione nel file sorgente. Ecco come si procede:

```
class Example {  
public:  
    void Method();  
};
```

```
#include "Example.h"

void Example::Method()
{
    std::cout << "Method was called" << std::endl;
}
```

Non è obbligatorio creare due file distinti per questo scopo; è possibile ottenere lo stesso risultato all'interno di un unico file e, in alcune situazioni, ciò risulta vantaggioso.

```
#include <iostream>

class Example {
public:
    void Method();
};

void Example::Method() { std::cout << "Method was called" <<
std::endl; }
```

Sentiti libero di affrontare il compito utilizzando il metodo che preferisci. Tuttavia, una buona pratica comune è **separare dichiarazione e implementazione**.

Membri Statici della Classe

Nella programmazione orientata agli oggetti, la parola chiave `static` assume un significato particolare, modificando il comportamento di attributi e metodi. Esistono situazioni in cui è necessario che alcuni dati siano condivisi tra tutti gli **oggetti** di una **classe** invece di essere replicati per ogni istanza. In questi casi entrano in gioco i **membri dati statici**.

Sintassi dei membri statici

La creazione di un **membro statico** di una **classe** è semplice. È sufficiente anteporre la dichiarazione con la parola chiave `static`.

```
class Example {  
public:  
    static int static_attribute;  
    static void static_method() { std::cout << "Static method!" <<  
std::endl; }  
};
```

Nell'esempio sopra, `static_attribute` e `static_method()` sono dichiarati come **membri dati statici** della `class Example`. A differenza dei membri dati regolari, i membri dati statici sono associati alla **classe** stessa piuttosto che ai singoli **oggetti**. Questo significa che tutte le istanze di `Example` condividono gli stessi **membri statici**.



Note

Per accedere a un membro dati statico, utilizziamo l'operatore di risoluzione dell'ambito (`::`). Ma prima di questo dobbiamo inizializzarlo.

L'inizializzazione è fondamentale per i membri dati statici e deve essere effettuata al di fuori della classe, a meno che il membro non utilizzi anche la parola chiave `const`.

```
class Example {
public:
    static int static_attribute;
    static void static_method() { std::cout << "Static method!" <<
std::endl; }
};

int Example::static_attribute = 0;
```

```
class ExampleConst {
public:
    static const int static_const_attribute = 0;
    static void static_method() { std::cout << "Static method!" <<
std::endl; }
};
```

Vantaggi dell'utilizzo dei membri statici

L'uso dei membri dati statici e delle funzioni membro statiche offre diversi vantaggi.

Caso d'uso	Riepilogo
Condivisione delle risorse	Condividi dati tra tutte le istanze della classe per risparmiare memoria.
Funzioni di utilità	Esegui logica non collegata allo stato specifico di un oggetto.
Operazioni a livello di classe	Agisci sui dati della classe senza creare oggetti.

```
#include <iostream>

class Example {
public:
    static int static_attribute;
    static void static_method() { std::cout << "Static method!" <<
std::endl; }
};

// Initialization of the static member variable
int Example::static_attribute = 0;

int main()
{
    Example obj1, obj2;
    obj1.static_attribute = 100; // Modifying static_attribute through
obj1

    std::cout << obj1.static_attribute << std::endl;
    std::cout << obj2.static_attribute << std::endl;

    Example::static_attribute = 25; // Modifying static_attribute
through class
    Example::static_method(); // Calling the static method through
class
}
```



Note

Prova a sperimentare con una `Example class` e i suoi **membri statici**.

La Parola Chiave This

La parola chiave `this` nella programmazione orientata agli oggetti si riferisce all'**istanza corrente dell'oggetto**. Fornisce alle funzioni membro un puntatore all'oggetto che le ha invocate, consentendo l'accesso alle variabili e alle funzioni di quell'oggetto.

```
#include <iostream>

class Example {
public:
    Example* get_address() { return this; }
};

int main()
{
    Example obj;
    std::cout << &obj << std::endl;
    std::cout << obj.get_address() << std::endl;
}
```

Quando si utilizza l'operatore punto (`.`) per accedere a variabili membro o funzioni membro all'interno di una **classe**, il puntatore `this` punta implicitamente all'**oggetto** per cui la funzione membro è chiamata. Questo è spesso utile in diversi scenari ed è comunemente impiegato per evitare ambiguità nei nomi delle variabili.

```
#include <iostream>

class Example {
public:
    void multiply(int number) { number *= number; }
    int number;
};

int main()
{
    Example obj;
    obj.number = 2;
    obj.multiply(5);
    std::cout << obj.number;
}
```

Qui possiamo utilizzare la parola chiave `this` poiché **punta all'oggetto corrente** come puntatore. Per accedere ai suoi attributi, è necessario utilizzare l'operatore `->`.



Note

Il valore dell'attributo non è cambiato perché il programma non distingue a quale numero ci si riferisce, dato che condividono lo stesso nome.

```
#include <iostream>

class Example {
public:
    void multiply(int number) { this->number *= number; }
    int number;
};

int main()
{
    Example obj;
    obj.number = 2;
    obj.multiply(5);
    std::cout << obj.number;
}
```

L'utilizzo della parola chiave `this` è una pratica diffusa in diversi scenari. Viene spesso impiegata, in particolare nelle **classi** di grandi dimensioni, per indicare che una variabile è un membro delle **classi** e non un parametro o argomento esterno.

Panoramica su Classi e Oggetti

Cos'è una Classe?

Una **classe** può essere considerata come un modello per la creazione di qualcosa.

Definisce la struttura e il comportamento di ciò che si andrà a creare, che in termini di programmazione viene chiamato **oggetto**.

```
class Cake {  
public:  
    // Attributes  
    float flour;  
    std::string filling;  
  
    // Methods  
    void bake() {  
        // working with attributes  
    }  
};
```

- **Attributi:** noti anche come membri dati, sono le proprietà che definiscono lo stato.
- **Metodi:** noti anche come membri funzione, sono le funzioni o azioni che possono essere eseguite.



Note

Immagina una classe come una ricetta per una torta. La ricetta specifica quali ingredienti e passaggi sono necessari per preparare la torta, ma non è la torta stessa.

Che cos'è un Oggetto?

Un **oggetto** è un'istanza di una **classe**. Quando una **classe** viene definita, non viene allocata memoria fino a quando non viene creata un'istanza di essa. Un **oggetto** rappresenta un esempio specifico o un'istanza di una classe, con **valori effettivi assegnati ai suoi attributi**.



Note

Continuando con l'analogia della torta, un oggetto è la torta reale creata utilizzando la ricetta (classe), e ogni torta sarà unica in base alle sue caratteristiche.

Relazione tra Classi e Oggetti

Gli oggetti vengono creati a partire da una classe e ciascun oggetto possiede una propria identità, stato e comportamento. Le classi e gli oggetti hanno una relazione fondamentale in cui:

- Le **classi** definiscono la struttura e le capacità.
- Gli **oggetti** sono istanze specifiche che incarnano la definizione della classe.

```
int main()
{
    // Creating an object of the Cake class
    Cake myCake;

    // Setting attributes
    myCake.flour = 25.1f;
    myCake.filling = "strawberry";

    // Calling a method
    myCake.bake();
}
```

Costruttori e Distruttori

Che cos'è un costruttore della classe



Definizione

Il **costruttore** è un metodo speciale di una **classe** che viene chiamato quando un oggetto viene creato. Con esso è possibile **inizializzare gli attributi con valori significativi**.

Utilizzando i costruttori, è possibile semplificare la creazione degli oggetti. Senza un costruttore, sarebbe necessario **impostare ogni proprietà singolarmente** e potrebbe essere necessario chiamare più metodi solo per rendere l'oggetto utilizzabile.

Lo scopo principale di un costruttore è **inizializzare l'istanza appena creata**. Può impostare i valori iniziali degli attributi **in base ai parametri passati al costruttore**, garantendo che l'oggetto inizi in uno stato valido.

Sintassi del costruttore

Sebbene il processo di sviluppo di un **costruttore** possa sembrare semplice, coinvolge numerose considerazioni specifiche. L'approccio generale per crearne uno è:

```
class Student {  
public:  
    Student() {  
        name = 'Default';  
    }  
  
    std::string name;  
};
```

- **Nome:** il costruttore ha lo stesso nome della classe;
- **Tipo di ritorno:** i costruttori non hanno un tipo di ritorno, nemmeno `void`;
- **Parametri:** i costruttori possono accettare parametri e possono essere sovraccaricati con diversi insiemi di parametri per consentire vari modi di inizializzare gli oggetti;
- **Invocazione automatica:** il costruttore viene chiamato automaticamente dal compilatore quando viene creato un oggetto.

Costruttore di default

Un **costruttore di default** è un costruttore che può essere invocato senza argomenti. Ecco come vengono definiti e utilizzati i costruttori di default:

```
#include <iostream>

class Example {
public:
    Example()
    {
        std::cout << "Object was created!";
    }
};

int main()
{
    Example obj;
}
```

Puoi sperimentare con il codice sopra. La logica scritta nel metodo **Example()** viene eseguita **ogni volta** che viene creata un'istanza della *classe*.



Nota

Prova ad aggiungere un altro **oggetto** o a modificare la logica all'interno del **costruttore**.

Se non viene definito alcun costruttore, il compilatore ne genera uno predefinito. Il suo comportamento dipende dai tipi delle variabili membro: tipi fondamentali, tipi con costruttore predefinito o tipi definiti dall'utente.

```
#include <iostream>

class Example {
public:
    int value;
};

int main()
{
    Example obj; // Default constructor
    std::cout << obj.value;
}
```

Un attributo è un tipo di dato fondamentale. Il costruttore predefinito generato automaticamente **non inizializza le variabili membro di tipo incorporato**. Queste avranno **valori indeterminati** se si crea un oggetto senza inizializzare esplicitamente questi campi.



Nota

Potresti notare che il compilatore fornisce un valore coerente per la variabile **value**, ma lo standard **non lo richiede** ed è possibile che contenga un valore spazzatura.

Costruttore e Attributi

Uno degli utilizzi principali dei costruttori è **inizializzare gli attributi della classe**. Un **costruttore di default**, ad esempio, può essere utilizzato per impostare valori iniziali. Ad esempio:

```
#include <iostream>

class Person {
public:
    Person() { name = "undefined"; }

    std::string name;
};

int main()
{
    Person person;
    std::cout << person.name;
}
```

Se non si specifica un valore per l'**attributo name** dell'**oggetto**, verrà impostato su **undefined** come **valore predefinito**. È possibile provare a rimuovere questo costruttore per osservare quali cambiamenti si verificano.

Inizializzazione degli attributi con il costruttore

Come le funzioni, i costruttori possono accettare parametri, consentendo diversi argomenti durante la creazione dell'oggetto. È inoltre possibile sovraccargarli per gestire un numero variabile di argomenti.

```
class Person {
public:
    Person(std::string _name, std::string _surname, int _age)
    {
        name = _name;
        surname = _surname;
        age = _age;
    }

    std::string name;
    std::string surname;
    int age;
};

int main()
{
    Person person("Bob", "Song", 23);
}
```

Lista di Inizializzazione del Costruttore

Una **lista di inizializzazione** permette ai costruttori di inizializzare i membri della classe **prima che venga eseguito il corpo del costruttore**. È particolarmente utile per inizializzazioni complesse o quando si lavora con **oggetti** di **classi** che hanno membri `const`.

Sintassi della Lista di Inizializzazione

La sintassi delle **liste di inizializzazione** può sembrare eccessivamente complessa e confusa. Si potrebbe pensare di utilizzare solo un costruttore come alternativa più semplice. Tuttavia, man mano che il programma cresce, le liste di inizializzazione **si dimostrano comode e semplici da utilizzare**.

- `:` (**simbolo due punti**): introduce la lista di inizializzazione dei membri in un costruttore;
- **member (value)**: una variabile membro della classe che viene inizializzata con il valore (argomento passato al costruttore);
- `{ }` (**parentesi graffe**): il corpo del costruttore, dove è possibile eseguire codice aggiuntivo dopo l'inizializzazione dei membri.

```
class TemperatureSensor {
public:
    TemperatureSensor(int initialTemperature)
        : currentTemperature(initialTemperature) {}

    int currentTemperature;
};
```

Limitazioni e Considerazioni

Esiste un malinteso comune riguardo l'ordine di inizializzazione nella **lista di inizializzazione**. Contrariamente a quanto si crede, l'inizializzazione delle variabili nella lista di inizializzazione **non segue l'ordine in cui sono scritte**. Invece, è **determinato dall'ordine di dichiarazione dei membri nella classe**, e non dalla sequenza nella lista di inizializzazione.

```
#include <iostream>

class Exchanger {
public:
    Exchanger(float _quantity, float _rate)
        : quantity(_quantity), rate(_rate), total(quantity * rate) {}

    float quantity, rate, total;
};

int main()
{
    Exchanger exchanger(100, 0.3);
    std::cout << exchanger.total;
}
```



Nota

Prova a modificare la sequenza di inizializzazione della variabile `total` nella lista di inizializzazione e nella dichiarazione della variabile all'interno della `class`.

Quando si utilizzano le liste di inizializzazione in un costruttore, l'argomento può avere lo stesso nome dell'attributo della `class`.

```
#include <iostream>

class Exchanger {
public:
    Exchanger(float quantity, float rate)
        : quantity(quantity), rate(rate), total(quantity * rate) {}

    float quantity, rate, total;
};

int main()
{
    Exchanger exchanger(100, 0.3);
    std::cout << exchanger.total;
}
```

Ma se si utilizza lo stesso nome sia per le variabili membro che per gli argomenti, `total` assumerà il valore **dei parametri moltiplicati, non dei membri**. Per verificarlo, prova a impostare `quantity` a zero invece che all'argomento.

Delegazione del Costruttore

Delegazione del costruttore (nota anche come concatenazione o inoltro del costruttore) consente a un costruttore di chiamarne un altro all'interno della stessa **classe** per riutilizzare la logica di inizializzazione comune.

Sintassi della delegazione del costruttore

La **delegazione del costruttore** viene solitamente utilizzata all'interno della sintassi di una lista di inizializzazione. Prevede l'uso dell'operatore due punti (`:`), seguito dal costruttore a cui si desidera delegare e da eventuali argomenti o parametri aggiuntivi da passare.

```
class UserAccount {
public:
    // Primary constructor
    UserAccount(int userId, int accessLevel)
        : userId(userId), accessLevel(accessLevel)
    {
        // Constructor body
    }

    // Delegating constructor (default accessLevel)
    UserAccount(int userId) : UserAccount(userId, 0) {
        // Constructor body
    }

    int userId;
    int accessLevel;
};
```

L'uso delle **liste di inizializzazione** per la delega dei costruttori non è obbligatorio, ma è generalmente **consigliato** per chiarezza e prestazioni. Se necessario, è possibile chiamare un **costruttore sovraccarico** da un altro costruttore.

```
#include <iostream>

class Point {
public:
    // Delegating constructor to initialize default point at origin
    Point() {
        Point(0, 0);
    }

    // Main constructor
    Point(int x, int y)
        : x(x), y(y) {}

    int x, y;
};
```



Nota

Potenziale ricorsione infinita può verificarsi durante l'utilizzo della delega dei costruttori. Assicurarsi che i costruttori siano strutturati per evitare cicli di invocazione ricorsiva

La **delega dei costruttori** offre molteplici vantaggi nella programmazione orientata agli oggetti ed è comoda da utilizzare, nonostante la complessità iniziale che potrebbe sembrare avere.

Che cos'è un distruttore della classe



Definizione

I **distruttori** sono funzioni membro speciali che vengono invocate quando termina la vita di un `object's`. Il loro scopo principale è rilasciare le risorse che l'oggetto potrebbe aver acquisito durante la sua esistenza.

Sintassi del distruttore

Anche se hanno scopi opposti, creare un distruttore è molto simile a creare un costruttore. Condividono quasi gli stessi principi di dichiarazione. L'approccio generale per crearne uno è:

- **Nome:** ha lo stesso nome della classe, ma è preceduto da una tilde (~);
- **Nessun tipo di ritorno:** non ha un tipo di ritorno, nemmeno void;
- **Nessun parametro:** non può accettare parametri;
- **Invocazione automatica:** viene chiamato automaticamente dal compilatore quando l'oggetto esce dallo scope o viene eliminato esplicitamente.

Invocazione automatica dei distruttori

L'**invocazione automatica dei distruttori** si riferisce alla chiamata automatica del distruttore. Questo garantisce che le risorse detenute dall'oggetto vengano rilasciate correttamente, contribuendo così a prevenire perdite di memoria e di risorse.

```
int main()
{
    {
        Example obj; // Object created
    } // Object goes out of scope here, destructor is invoked
}
```

```
int main()
{
    Example *obj = new Example(); // Dynamically allocated object
    delete obj; // Destructor is invoked before memory is deallocated
}
```

```
int main()
{
    Example obj; // Global or static object
    return 0; // Destructor is invoked before program terminates
}
```

- quando un oggetto **esce dall'ambito**;
- quando l'operatore `delete` viene utilizzato per eliminare **oggetti allocati dinamicamente**;
- quando il programma **termina**.

Necessità dei Distruttori

Lo scopo principale di un distruttore è **rilasciare le risorse acquisite dall'oggetto durante il suo ciclo di vita**. Questo include la chiusura di handle di file, la deallocazione della memoria (utilizzando `new`) o compiti simili.

```
class ResourceHolder {
public:
    DynamicArray(int size) {
        data = new int(100);
    }

    ~DynamicArray() {
        delete data;
    }
private:
    int* data;
};
```

```
class FileHandler {
public:
    FileHandler(const char* filename) {
        file.open(filename);
    }

    ~FileHandler() {
        file.close();
    }
private:
    std::ofstream file;
};
```

Panoramica sull'Incapsulamento

Introduzione all'Incapsulamento



Definizione

L'**incapsulamento** è un concetto fondamentale nella programmazione orientata agli oggetti che consiste nell'avvolgere dati e metodi all'interno di una classe. Questo principio serve a raggruppare i dati dell'oggetto e a proteggerli da accessi esterni e usi impropri, limitando l'accesso diretto ai suoi componenti.

Principi di base

L'idea centrale dell'incapsulamento è mantenere il funzionamento interno **nascosto dal mondo esterno**. Questo significa che gestisce il proprio stato internamente ed espone solo un'interfaccia definita per l'interazione con l'esterno.

Questa interfaccia di solito include metodi per lavorare con i valori e gli attributi delle proprietà dell'oggetto, mantenendo però tali proprietà `private`. In questo modo, l'incapsulamento garantisce che i dati dell'oggetto **possano essere modificati solo in modi prevedibili e controllati**.

Vantaggio	Descrizione
Sicurezza migliorata	Protegge i dati nascondendo gli stati interni degli oggetti.
Facilità di manutenzione	Le modifiche all'interno di una classe non influenzano altre parti.
Complessità ridotta	Mantiene dati e comportamenti organizzati e modulari.
Modifica controllata	Consente modifiche sicure e prevedibili agli oggetti.

Parole Chiave Dei Modificatori Di Accesso

L'incapsulamento in C++ si ottiene principalmente tramite l'uso dei **modificatori di accesso**. Questi modificatori controllano il livello di accesso che le altre parti del programma hanno ai membri (sia dati che funzioni) di una classe. I tre principali modificatori di accesso sono:

- **private** : accessibile solo all'interno della stessa classe e nascosto all'esterno della classe. Questo è il livello di accesso predefinito per i membri di una classe;
- **protected** : accessibile all'interno della classe e delle sue classi derivate. Sono più accessibili dei membri **private** ma forniscono comunque un certo livello di protezione dei dati;
- **public** : accessibile da qualsiasi parte del programma. Sebbene i membri **public** non siano incapsulati, sono essenziali per definire l'interfaccia che la classe espone alle entità esterne.



Nota

Il **modificatore di accesso** continua ad applicarsi fino a quando non ne viene specificato o incontrato un altro.

```
#include <iostream>

class Person {
public:
    std::string name;
};

int main()
{
    Person person;
    person.name = "Bob";
    std::cout << person.name;
}
```



Nota

Prova a eliminare o modificare la parola chiave `public` e osserva cosa succede.

Analogamente a come un conducente può guidare un'auto **senza comprendere la meccanica interna**, utenti e programmatori non devono preoccuparsi degli attributi e dei metodi `private` all'interno di una classe. Seguire queste regole:

- Mantenere i membri dati `private` o `protected`;
- Fornire metodi `public` per accedere e modificare i dati `private`;
- Garantire che questi metodi svolgano solo ciò per cui sono stati progettati, **senza rivelare la logica interna**.

Una `class` ben incapsulata dovrebbe esporre solo **ciò che è necessario agli utenti** e nascondere il proprio stato interno e i dettagli di implementazione.

Metodi Accessor e Mutator

L'incapsulamento implica anche la restrizione dell'accesso diretto ad alcuni componenti di un oggetto, ed è qui che entrano in gioco le funzioni accessor e mutator. Le funzioni **accessor e mutator**, più comunemente note come **getter e setter**, sono metodi `public` che forniscono un accesso controllato ai membri dati `private` di una classe.

Le **funzioni accessor (getter)** sono funzioni che permettono di leggere i valori dei membri dati privati senza modificarli. Sono fondamentali per ottenere lo stato di un oggetto mantenendo i membri dati nascosti e protetti.

```
class Example {  
public:  
    int get_member() { return member; }  
private:  
    int member;  
};
```

Funzioni Mutatorie (Setters) sono funzioni che permettono la modifica dei valori dei membri dati privati. Offrono un modo controllato per cambiare lo stato di un oggetto. Utilizzando i setters, è possibile implementare logiche di validazione per garantire che solo dati validi vengano assegnati ai membri dati.

```
class Example {  
public:  
    void set_member(int value) { member = value; }  
private:  
    int member;  
};
```

La funzione principale dei getters e setters è gestire l'accesso ai membri di una classe, riducendo così la probabilità di errori causati dalla manipolazione diretta. Ad esempio, consentono di limitare l'assegnazione di valori eccessivamente grandi a determinate proprietà. È possibile limitare la potenza del riscaldatore al valore `10`; non è possibile impostarla oltre tale valore.

```
#include <iostream>

class Heater {
public:
    void setPower(int value) { power = value > 10 ? 10: value; }
    int getPower() { return power; }
private:
    int power;
};

int main()
{
    Heater heater;
    heater.setPower(7);
    std::cout << heater.getPower();
}
```

Segmentazione e Aggregazione

Quando si crea un software che sia manutenibile e flessibile, ci sono molti aspetti da considerare nel processo, specialmente nella programmazione orientata agli oggetti. **Composizione** e **aggregazione** sono concetti fondamentali che permettono la **creazione di sistemi complessi a partire da componenti più semplici** e fanno parte del paradigma dell'incapsulamento.

Composizione

La composizione è una tecnica di progettazione in cui una **class** contiene **objects** di altre **classes** come variabili membro. Questi membri sono componenti essenziali della **class** contenitore e presentano una forte relazione di possesso. Quando l'**object** contenitore viene distrutto, anche le sue parti composte vengono distrutte. Ad esempio:

```
#include "Engine.h"

class Car {
public:
    void Start()
    {
        engine.Start(); // Start the engine
        std::cout << "Car started" << std::endl;
    }
private:
    Engine engine; // Engine object composition
};
```

```
class Engine {  
public:  
    void Start()  
    {  
        std::cout << "Engine started" << std::endl;  
    }  
};
```

La classe `Car` compone un oggetto `Engine`. Quando viene creato un oggetto `Car`, viene automaticamente creata un'istanza di `Engine` come parte della sua composizione.

Aggregazione

Aggregazione è un'altra forma di composizione di `object` in cui una `class` contiene `object` di altre `classes`, ma le istanze contenute hanno una relazione più debole rispetto alla composizione. Nell'aggregazione, le `classes` membri contenute **possono esistere indipendentemente** e possono essere condivise tra più `classes`.

```
#include "Engine.h"

class Car {
public:
    Car(Engine* engine)
        : engine(engine) {}

    void Start()
    {
        engine->Start(); // Start the engine
        std::cout << "Car started" << std::endl;
    }
private:
    Engine* engine; // Engine object aggregation
};
```

```
class Engine {
public:
    void Start()
    {
        std::cout << "Engine started\n";
    }
};
```

La classe `Car` aggrega un oggetto `Engine` utilizzando un puntatore. La classe `Car` non possiede l'oggetto `Engine`; si limita a mantenere un riferimento ad esso. Questo permette all'istanza di `Engine` di esistere indipendentemente dall'oggetto `Car` e di essere condivisa tra più istanze se necessario.

Scelta tra Composizione e Aggregazione

Durante la progettazione di sistemi software, è fondamentale valutare attentamente se utilizzare la composizione o l'aggregazione in base alle relazioni tra classi e oggetti.

	Composizione	Aggregazione
<code>class</code> possiede i suoi <code>objects</code>	Sì	No
<code>objects</code> sono parte integrante <code>part</code>	Sì	No
<code>class</code> fa solo riferimento a <code>objects</code>	No	Sì
<code>objects</code> possono essere condivisi	No	Sì

Nel contesto delle classi `Car` e `Engine`, l'utilizzo della **composizione** sarebbe più appropriato. Ogni auto di solito ha il proprio motore, e il motore è **una parte integrante dell'auto stessa**. Inoltre, ciò impedisce che l'oggetto `Engine` venga condiviso o riutilizzato tra più oggetti `objects`, il che risulta coerente.

La Parola Chiave Friend

La parola chiave `friend` rappresenta un costrutto unico, offrendo una deviazione dai principi standard di incapsulamento della programmazione orientata agli oggetti. Consente a una funzione o a un'altra `class` di accedere ai membri `private` e `protected` di una `class`.

```
#include <iostream>

class Example {
private:
    const static int static_private_member = 0;
    friend void func(); // grants access to the private members to
func()
};

void func()
{
    std::cout << Example::static_private_member;
}

int main()
{
    func();
}
```

L'utilizzo di questo approccio viola l'incapsulamento poiché permette a entità esterne di accedere ai membri della `class`. Tuttavia, possono esserci situazioni in cui ciò è necessario. Ad esempio:

```
class KeyManager {
public:
    KeyManager(const std::string& key) : encryptionKey(key) {}
private:
    std::string encryptionKey;
};
```

La **encryptionKey** viene mantenuta **private** e non viene fornito alcun metodo di accesso perché si desidera **impedire l'accesso esterno da fuori** la **class**. Tuttavia, se si **presenta la necessità di utilizzare un algoritmo esterno** per la cifratura e la decifratura, entra in gioco la parola chiave friend **keyword**.

```
#include "CryptographicAlgorithm.h"

class KeyManager {
public:
    KeyManager(const std::string& key) : encryptionKey(key) {}
private:
    std::string encryptionKey;
    // Allow CryptographicAlgorithm access to private members
    friend class CryptographicAlgorithm;
};
```

```
class CryptographicAlgorithm {
public:
    std::string encrypt(const std::string& data, KeyManager&
keyManager);
    std::string decrypt(const std::string& encryptedData, KeyManager&
keyManager);
};
```

L'uso più comune della parola chiave `friend` si presenta quando sono necessari **interventi rapidi**, con l'intenzione di effettuare una successiva rifattorizzazione. È preferibile progettare le relazioni tra le `class` senza farvi affidamento, anche se possono comunque verificarsi scenari specifici.

Panoramica sull'Ereditarietà

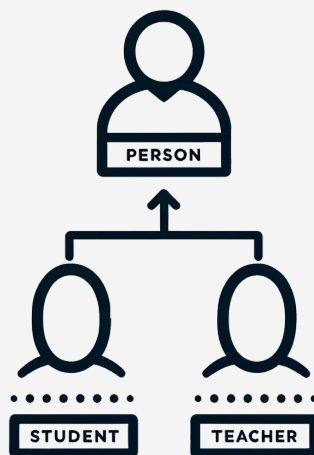
Introduzione all'Ereditarietà



Definizione

Ereditarietà è un concetto fondamentale nella programmazione orientata agli oggetti che consente a una nuova `class` di ereditare proprietà e comportamenti da una già esistente. Attraverso l'ereditarietà, la nuova `class`, spesso chiamata **sottoclasse**, acquisisce gli attributi e i metodi della classe esistente, nota come **superclasse**.

Per comprendere meglio i concetti di ereditarietà, esaminiamo alcuni esempi pratici. Il principale vantaggio dell'ereditarietà è il **riutilizzo del codice**. Ad esempio, le classi `Student` e `Teacher` riutilizzano il codice dalla classe `Person` (`name`, `age` e il metodo `display_info`). Questo **riduce la ridondanza** e rende il codice **più manutenibile**.



Sintassi dell'ereditarietà

Una classe derivata viene dichiarata utilizzando una dichiarazione di classe che specifica la classe base da cui eredita. Questo viene fatto utilizzando i due punti seguiti dallo specificatore di accesso (`public` , `protected` o `private`) e dal nome della classe base.

```
class Base {  
    // Base class members  
};  
  
class Derived : public Base {  
    // Derived class members  
};
```

- **Classe base (superclasse):** la `class` le cui proprietà e funzioni vengono ereditate. È anche conosciuta come classe genitore o superclasse.
- **Classe derivata (sottoclasse):** la `class` che eredita dalla classe genitore. È anche conosciuta come classe figlia o sottoclasse.
- **Specificatore di accesso:** questo specificatore determina come i membri della classe base vengono ereditati dalla classe derivata.
- **Ereditarietà:** i due punti (`:`) seguiti dallo `accessSpecifier` e dalla `BaseClass` indicano che la `DerivedClass` sta ereditando dalla `BaseClass`.

Tipi di ereditarietà

Esistono diversi tipi di ereditarietà. Ogni tipo offre un modo unico di stabilire relazioni tra classi, fornendo così una base per una progettazione orientata agli oggetti efficace. Ecco i principali tipi di ereditarietà:

Tipo	Descrizione
Ereditarietà Singola	Coinvolge una sola classe base e una sola classe derivata.
Ereditarietà Multipla	Una classe derivata eredita da più di una classe base.
Ereditarietà Multilivello	Comprende una catena di classi in cui una classe è derivata da un'altra classe derivata.
Ereditarietà Gerarchica	Più classi sono derivate da una singola classe base.
Ereditarietà Ibrida	Una combinazione di due o più tipi di ereditarietà.

 Modificatori di Accesso nell'Ereditarietà

I **modificatori di accesso** svolgono un ruolo fondamentale nella programmazione orientata agli oggetti, in particolare nell'ereditarietà. Essi determinano come i membri (**attributi e metodi**) di una classe base possono essere accessibili dalle classi derivate. Comprendere questi modificatori è essenziale per progettare classi efficaci e mantenere l'incapsulamento e l'integrità dei dati.

Tipi di accesso nell'ereditarietà

Una classe può essere derivata da un'altra classe. La classe derivata eredita i membri dalla classe base, ma l'accessibilità di questi membri ereditati dipende sia dal modificatore di accesso utilizzato nella classe base sia dal tipo di ereditarietà.

	Base Class public	Base Class protected	Base Class private
<code>public</code>	Public in Derived Class	Protected in Derived Class	Not Accessible
<code>protected</code>	Protected in Derived Class	Protected in Derived Class	Not Accessible
<code>private</code>	Private in Derived Class	Private in Derived Class	Not Accessible

```
class Derived : public Base {  
    // publicMember is public  
    // protectedMember is protected  
    // privateMember is not accessible  
};
```

```
class Derived : protected Base {  
    // publicMember is protected  
    // protectedMember is protected  
    // privateMember is not accessible  
};
```

```
class Derived : private Base {  
    // publicMember is private  
    // protectedMember is private  
    // privateMember is not accessible  
};
```

Conclusione sul Controllo degli Accessi e l'Ereditarietà

Nell'ereditarietà orientata agli oggetti, i membri `private` di una classe base non sono accessibili alle classi derivate, proteggendoli da modifiche o letture. I membri `Protected` possono essere accessibili solo all'interno della **sottoclasse**, mentre i membri `public` sono accessibili anche dall'esterno. È possibile sperimentare questo comportamento utilizzando il frammento di codice seguente.

```
class Base
{
public:
    int publicAttribute;
protected:
    int protectedAttribute;
private:
    int privateAttribute;
};

class PublicDerived : public Base {};
class ProtectedDerived : protected Base {};
class PrivateDerived : private Base {};

int main()
{
    PublicDerived obj1;
    ProtectedDerived obj2;
    PrivateDerived obj3;
}
```



Nota

I membri `Protected`, accessibili all'interno delle classi derivate e ulteriormente derivate, fungono da collegamento tra gli elementi `private` e `public`.

I costruttori e i distruttori vengono invocati automaticamente per gli oggetti della classe derivata, garantendo una corretta inizializzazione e pulizia delle risorse. Per accedere direttamente a questi elementi della classe base, i costruttori e i distruttori devono essere dichiarati `public`.

Ereditarietà Multipla

L'ereditarietà multipla consente a una classe derivata di ereditare da più classi base. Ciò significa che una singola classe derivata può accedere ai membri (dati e funzioni) di più classi base, combinando di fatto i loro attributi e comportamenti.

Ereditarietà Multipla Orizzontale

In questo tipo di ereditarietà multipla, una **classe** eredita proprietà e metodi da più **superclassi** allo stesso livello nella gerarchia di ereditarietà. Si considerino le **classes**: **Shape** e **Color**, ciascuna con proprietà e metodi distinti. La **Shape** definisce la forma di un **oggetto**, mentre la **Color** ne definisce il colore.

```
class Shape {
public:
    void setShape(const std::string& value) { shape = value; }
    std::string getShape() { return shape; }
private:
    std::string shape;
};
```

```
class Color {
public:
    void setColor(const std::string& value) { color = value; }
    std::string getColor() { return color; }
private:
    std::string color;
};
```

Ora, creiamo una **sottoclasse** chiamata **ColoredShape** che eredita proprietà e metodi sia dalla classe **Shape** che dalla classe **Color**.

```
#include "Shape.h"
#include "Color.h"

class ColoredShape : public Shape, public Color {
public:
    void describe() {
        std::cout << "This object is a " << getShape() << " shape and
is " << getColor() << '\n';
    }
};
```

Ereditarietà verticale

Nell'ereditarietà in profondità, una `class` eredita proprietà e metodi dal suo **genitore diretto e dai suoi antenati**, formando una catena di ereditarietà. Ad esempio, si consideri la `class Vehicle`, che può servire come base per l'ereditarietà di `Car`, `Truck` e altri. Nel nostro esempio, utilizzeremo `Car` per illustrare il concetto.

```
class Vehicle {
public:
    void start() { std::cout << "Vehicle started"; }
    void stop() { std::cout << "Vehicle stopped"; }
};
```

```
#include "Vehicle.h"

class Car : public Vehicle {
public:
    void drive() { std::cout << "Car is being driven"; }
};
```

Per ottenere l'**ereditarietà verticale**, è necessario impostare una gerarchia in cui una `class` eredita da un'altra, e una classe successiva eredita dalla prima, e così via. Possiamo creare una `ElectricCar` che eredita tutte le proprietà e funzionalità dalla `Car`, la quale a sua volta eredita dalla `Vehicle`, stabilendo così una struttura di **ereditarietà multipla complessa**.

```
#include "Car.h"

class ElectricCar : public Car {
public:
    void charge() { std::cout << "Electric car is charging"; }
};
```

Perché è Necessaria l'Ereditarietà Multipla

L'**ereditarietà multipla** offre **flessibilità e riutilizzo del codice** in situazioni in cui una `class` deve mostrare comportamenti o caratteristiche di più di una `class` genitore. Ecco alcuni scenari in cui l'ereditarietà multipla è vantaggiosa:

- **Ruoli del mondo reale:** un uccello volante può combinare le caratteristiche sia delle classi `Flying` che `Bird`, rappresentando entrambe le abilità. La classe `Flying` può essere applicata anche ad aerei o altri oggetti volanti.
- **Riutilizzo del codice:** l'ereditarietà multipla consente a una classe di riutilizzare funzionalità da diverse classi base senza duplicazione.
- **Interfacce focalizzate:** incoraggia la combinazione di interfacce piccole e specifiche invece di utilizzarne una grande e generale.



Nota

Scomporre un **oggetto** complesso in oggetti più semplici e utilizzare l'ereditarietà multipla per creare software **flessibile e manutenibile**.

Costruttore e Distruttore nell'Ereditarietà

Prima il Costruttore della Classe Base

Nel contesto dell'ereditarietà, i costruttori svolgono un ruolo fondamentale nell'inizializzare correttamente le *classi* derivate. Comprendere **la sequenza con cui vengono invocati i costruttori è essenziale** per cogliere le dinamiche dell'ereditarietà. Osserva l'output del frammento di codice qui sotto per vedere l'ordine delle chiamate ai costruttori.

```
#include <iostream>

class Base {
public:
    Base() { std::cout << "Base constructor called" << std::endl; }
};

class Derived : public Base {
public:
    Derived() { std::cout << "Derived constructor called" <<
std::endl; }
};

int main()
{
    Derived derivedObj;
}
```



Nota

Una volta che il costruttore della **Classe Base** ha completato la sua inizializzazione, viene eseguito il costruttore della **Classe Derivata**.

La **superclasse** viene chiamata per prima perché deve **inizializzare i membri ereditati** della sottoclasse. Questo garantisce che la **sottoclasse** inizi con uno stato valido e possa fare affidamento sullo stato inizializzato della sua classe base.

```
class Base {
public:
    Base(int value)
        : data(value) {}
private:
    int data;
};

class Derived : public Base {
public:
    Derived(int value)
        : Base(value) {}
};

int main()
{

}
```

Nell'esempio, il costruttore viene chiamato con parametri nella **lista di inizializzazione**. È necessario chiamare esplicitamente il costruttore della **superclasse** all'interno della lista di inizializzazione del costruttore della **sottoclasse**. Se non si specifica un costruttore della classe base nella lista di inizializzazione, il **costruttore di default** della **superclasse** viene chiamato automaticamente.

Distruttore della Classe Derivata per Primo

Quando un **oggetto** viene distrutto, i distruttori vengono invocati **nell'ordine inverso rispetto ai loro costruttori**. Questo significa che i distruttori vengono chiamati prima per la classe più derivata, poi per ciascuna classe base in ordine inverso rispetto alla loro dichiarazione.

```
#include <iostream>

class Base {
public:
    ~Base() { std::cout << "Base destructor called" << std::endl; }
};

class Derived : public Base {
public:
    ~Derived() { std::cout << "Derived destructor called" <<
std::endl; }
};

int main()
{
    Derived derivedObj;
}
```



Nota

Solo dopo che il distruttore della **Classe Derivata** ha completato la sua pulizia, viene invocato il distruttore della **Classe Base**.

Ereditarietà a Diamante

L'ereditarietà multipla può portare a una situazione nota come **problema del diamante** o **ereditarietà a diamante**, che rappresenta una sfida significativa nei linguaggi di programmazione orientati agli oggetti che supportano l'ereditarietà multipla.

Questo si verifica quando una **sottoclasse** eredita da due o più **classes**, le quali a loro volta ereditano da una **superclasse** comune. Il termine **diamante** viene utilizzato perché lo schema di ereditarietà ricorda la forma di un diamante.

```
class Base {};  
  
class Derived1 : public Base {};  
  
class Derived2 : public Base {};  
  
class Diamond : public Derived1, public Derived2 {};
```

Il problema principale dell'**ereditarietà a diamante** è l'**ambiguità** che crea. Poiché **Diamond** eredita sia da **Derived1** che da **Derived2**, che a loro volta ereditano da **Base**, esistono due copie di **Base** all'interno di un **oggetto** di tipo **Diamond**. Questo può portare a **ambiguità**.

```
#include <iostream>

class Base {
public:
    void display() { std::cout << "Base display()" << std::endl; }
};

class Derived1 : public Base { };
class Derived2 : public Base { };
class Diamond : public Derived1, public Derived2 { };

int main()
{
    Diamond obj;
    obj.display(); // Ambiguity: Which display() method should be
called?
}
```

Soluzione al problema

La parola chiave **virtual** aiuta a evitare questo problema. È possibile risolvere questa ambiguità tramite l'ereditarietà virtuale utilizzando la parola chiave **virtual**. Quando **Diamond** eredita virtualmente, C++ garantisce che sia presente una sola copia della **superclasse**, anche se viene ereditata più volte attraverso percorsi diversi.

```
class Base {};  
  
class Derived1 : virtual public Base {};  
  
class Derived2 : virtual public Base {};  
  
class Diamond : public Derived1, public Derived2 {};
```



Nota

Cerca di risolvere il problema di ambiguità nell'esempio precedente utilizzando l'ereditarietà virtuale.

Implementazione dell'Ereditarietà a Diamante

Per implementare efficacemente l'ereditarietà a diamante, utilizzare la **parola chiave virtual** nella dichiarazione della **superclasse** all'interno delle **classes** intermedie. Garantire un **uso coerente dell'ereditarietà virtuale** in tutti i percorsi della gerarchia di ereditarietà e prestare attenzione all'**ordine di chiamata** di costruttori e distruttori.

Risoluzione delle Ambiguità

Una delle sfide dell'ereditarietà multipla è anche la gestione delle potenziali ambiguità quando si hanno **membri con lo stesso nome**.

```
#include <iostream>

class Base {};

class Derived1 : public Base {
public:
    void display() { std::cout << "Derived1 display()" << std::endl; }
};

class Derived2 : public Base {
public:
    void display() { std::cout << "Derived2 display()" << std::endl; }
};

class Diamond : public Derived1, public Derived2 { };

int main() {
    Diamond obj;
    obj.display(); // Ambiguity: Which display() method should be
called?
}
```

Se entrambe le **superclassi** possiedono membri con lo stesso nome, la **sottoclasse** potrebbe non sapere quale utilizzare. Per risolvere tali ambiguità, è possibile utilizzare l'**operatore di risoluzione dello scope** (`::`) per specificare a quale membro della classe base si desidera accedere. Ad esempio:

```
#include <iostream>

class Base {};

class Derived1 : public Base {
public:
    void display() { std::cout << "Derived1 display()" << std::endl; }
};

class Derived2 : public Base {
public:
    void display() { std::cout << "Derived2 display()" << std::endl; }
};

class Diamond : public Derived1, public Derived2 { };

int main() {
    Diamond obj;
    obj.Derived1::display();
    obj.Derived2::display();
}
```

La Parola Chiave Final

La parola chiave `final` svolge un ruolo cruciale nella definizione delle `classes` e nel controllo delle gerarchie di ereditarietà. Fornisce agli sviluppatori un potente meccanismo per limitare determinati aspetti della progettazione delle `class`.

Utilizzo della parola chiave final

Quando una `class` viene dichiarata come `final`, indica che la `class` non può essere ulteriormente sottoposta a sottoclassificazione. In altre parole, rappresenta il punto finale della gerarchia di ereditarietà, impedendo qualsiasi ulteriore derivazione.

```
class Example final {  
    // Class members and methods  
};  
  
// cannot derive from final base Example  
class ExtendedExample: public Example {  
    // Class members and methods  
};
```

Questa funzionalità è particolarmente utile quando si desidera limitare la modifica o l'estensione di una specifica `class`, oppure quando si preferisce utilizzare la propria `class` tramite composizione piuttosto che tramite ereditarietà. Offre inoltre altri vantaggi, come una maggiore chiarezza del codice e la possibilità di alcune ottimizzazioni da parte del compilatore.

Beneficio	Descrizione
Controllo avanzato del design	Impedisce ulteriori ereditarietà, imponendo una gerarchia rigorosa.
Sicurezza del codice	Riduce i bug impedendo sottoclassificazioni rischiose.
Ottimizzazione delle prestazioni	Permette migliori ottimizzazioni da parte del compilatore.

Panoramica sul Polimorfismo

Introduzione al Polimorfismo



Definizione

Polimorfismo rappresenta un concetto fondamentale. Derivato dal greco che significa "molte forme", consente agli **oggetti** di diverse **classes** di essere trattati come un **oggetto** di una comune **superclasse**. L'aspetto più significativo è la capacità di entità diverse di rispondere ciascuna a modo proprio allo stesso messaggio o chiamata di metodo.

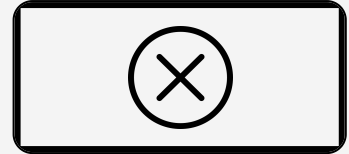
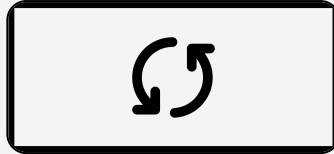
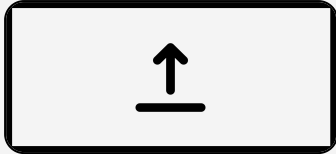
Tipi di polimorfismo

Esistono due principali tipi di polimorfismo: **a tempo di compilazione (statico)** e **a tempo di esecuzione (dinamico)**. Comprendere come e quando utilizzare ciascuno è fondamentale per scrivere codice flessibile ed efficiente.

- **Polimorfismo a tempo di compilazione:** si verifica tramite l'overloading di funzioni o operatori, dove il metodo da eseguire viene determinato durante la compilazione.
- **Polimorfismo a tempo di esecuzione:** utilizza funzioni virtuali, consentendo alle classi derivate di sovrascrivere i metodi della classe base, con la scelta del metodo corretto effettuata a runtime.

Applicazione e necessità del polimorfismo

Un ottimo modo per comprendere il polimorfismo è attraverso un'analogia reale. Si consideri un'interfaccia grafica con un pulsante. Questo pulsante può comportarsi in modo diverso a seconda del contesto—potrebbe funzionare come **pulsante di caricamento**, **pulsante di reset** o **pulsante di annullamento**.



Ogni pulsante esegue un'azione distinta quando viene cliccato, ma tutti essenzialmente fungono da **pulsanti**. Osserva l'implementazione teorica di questo concetto.

```
class UploadButton : public Button {
public:
    void onClick() { std::cout << "Upload" << std::endl; }
};
```

```
class ResetButton : public Button {
public:
    void onClick() { std::cout << "Reset" << std::endl; }
};
```

```
class CancelButton : public Button {
public:
    void onClick() { std::cout << "Cancel" << std::endl; }
};
```

Considerando che tutti i pulsanti condividono lo stesso metodo `onClick()` con implementazioni differenti, approfondiamo ulteriormente. E se avessimo bisogno di una funzione che accetti un **oggetto** appartenente a una delle **classes** di pulsanti come **parametro**?

```
void user_clicked_upload_button(const UploadButton& btn) {  
    btn.onClick();  
}  
  
void user_clicked_reset_button(const ResetButton& btn) {  
    btn.onClick();  
}  
  
void user_clicked_cancel_button(const CancelButton& btn) {  
    btn.onClick();  
}
```

Come puoi vedere, creare manualmente funzioni separate per ogni pulsante può **creare complessità**, specialmente durante le modifiche, poiché ogni funzione **deve essere modificata individualmente** se si presentano problemi. Inoltre, nella funzione principale, saranno necessari controlli aggiuntivi per determinare quale funzione chiamare. La **polimorfismo** consente di risolvere facilmente questi problemi.

Parole Chiave Virtual e Override

Parola chiave Virtual

La parola chiave `virtual` viene utilizzata prima della dichiarazione di un metodo in una `class` per risolvere il problema discusso in precedenza. Si impiega **per i metodi comuni a tutte le sottoclassi** ma che **richiedono implementazioni distinte per ciascuna**. In sostanza, appare come una normale dichiarazione di metodo, ad eccezione del fatto che inizia con `virtual`.

```
class Base {  
public:  
    virtual void display()  
    {  
        std::cout << "Base class display() called" << std::endl;  
    }  
};
```

Quando una funzione viene dichiarata come `virtual` in una **superclasse**, consente di essere sovrascritta in una **sottoclasse**. Questo significa che la sottoclasse può fornire la propria implementazione, che **verrà invocata al posto dell'implementazione originale** quando la funzione viene chiamata tramite un puntatore o riferimento alla **superclasse**.

```
#include <iostream>

class Base {
public:
    virtual void display()
    {
        std::cout << "Base display() called" << std::endl;
    }
};

class Derived : public Base {
public:
    void display()
    {
        std::cout << "Derived display() called" << std::endl;
    }
};

int main()
{
    Base* obj;
    Derived derived;

    obj = &derived;
    obj->display();
}
```



Nota

Quando si dispone di un puntatore a un **oggetto** e si desidera **accedere ai suoi membri** tramite il puntatore, si utilizza l'operatore freccia (`->`).

Ritornando alla soluzione per il problema originale con i pulsanti, questo metodo consente di **creare facilmente una singola funzione** che accetta un puntatore al `Button` padre come parametro. Di conseguenza, è possibile invocare questa funzione passando un `SubmitButton`, `ResetButton` o `CancelButton`, e si comporterà come previsto.

```
#include <iostream>

class Button {
public:
    virtual void onClick() { std::cout << "Click" << std::endl; }
};

class UploadButton : public Button {
public:
    void onClick() { std::cout << "Upload" << std::endl; }
};

class ResetButton : public Button {
public:
    void onClick() { std::cout << "Reset" << std::endl; }
};

class CancelButton : public Button {
public:
    void onClick() { std::cout << "Cancel" << std::endl; }
};

void user_clicked_button(Button& btn) {
    btn.onClick();
}

int main()
{
    UploadButton uploadButton;
    user_clicked_button(uploadButton);
}
```



Nota

Provare a creare **oggetti** di diverse **sottoclassi** e **passarli alla funzione**.

La parola chiave Override

La parola chiave `override` viene utilizzata in una **sottoclasse** per **indicare esplicitamente che una funzione membro esegue l'override di una funzione virtuale** proveniente da una **superclasse**. Aiuta a rilevare **errori** in fase di compilazione se la firma della funzione nella **sottoclasse** non corrisponde a nessuna funzione virtuale nella **superclasse**.

```
#include <iostream>

class Button {
public:
    virtual void handleClick() { std::cout << "Click" << std::endl; }
};

class UploadButton : public Button {
public:
    // marked override, but does not override
    void onClick() override { std::cout << "Upload" << std::endl; }
};

int main() { }
```



Se c'è un errore, come un errore di battitura nella firma della funzione o la **superclasse** non possiede la funzione che si sta tentando di sovrascrivere, **il compilatore genererà un errore**, aiutando a individuare potenziali **bug** in fase di compilazione. Questa funzionalità è particolarmente utile in ambienti di lavoro di grandi dimensioni, dove tenere traccia di modifiche come la rinomina dei metodi può essere complesso.

Metodo Virtuale Puro

Il concetto di **metodi virtuali** è fondamentale per ottenere il polimorfismo, consentendo un design software flessibile ed estensibile. I **metodi virtuali puri** ampliano questo concetto definendo interfacce e supportando il binding dinamico.

Sintassi del metodo virtuale puro

Un **metodo virtuale puro** viene dichiarato con la parola chiave `virtual` e **inizializzato a zero**. Indica che la funzione non ha una definizione all'interno della `class` e deve essere sovrascritta da qualsiasi **sottoclasse** prima che possano essere istanziati **oggetti** di quella `class`.

```
virtual void example() = 0;
```

Questo metodo funziona in modo simile a uno standard, tranne per il fatto che **non ha un corpo** racchiuso tra `{ }` ed è invece terminato con `= 0;`, indicando l'assenza di implementazione. Tuttavia, le altre regole per questo metodo **sono le stesse dei metodi regolari**.

Dichiarare un **metodo virtuale puro** rende la `class` in cui è dichiarato **astratta**, il che significa che diventa **impossibile istanziare oggetti** di quella classe. Questa restrizione esiste perché ogni metodo all'interno della `class` deve essere implementato prima di creare e utilizzare un'istanza, al fine di **prevenire errori e comportamenti imprevedibili**. Osserva il codice qui sotto:

```
class Animal {  
public:  
    virtual void speak() = 0;  
};
```

La `Animal` `class` presenta un **metodo virtuale puro** chiamato `speak()`, impedendo l'istanza di **oggetti** da questa classe. Questa progettazione è sensata, poiché lo scopo del metodo virtuale è **rappresentare i suoni distintivi** che ogni animale emette.

Rendendo il **metodo virtuale**, si consente a ogni **sottoclasse** di implementare la propria versione, cogliendo l'individualità del suono di ciascun animale. Dichiararlo come virtuale puro **indica che non esiste un'implementazione predefinita**, sottolineando la necessità di implementazioni concrete nelle classi derivate.

```
#include "Animal.h"

int main()
{
    // cannot declare variable 'animal' to be of abstract type
    Animal animal;
}
```

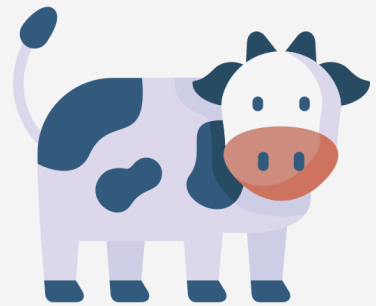
Anche questo è logico. Creare istanze della `Animal` `class` sarebbe **impraticabile e controintuitivo**, poiché rappresenta un **concetto astratto** che funge da categoria per vari animali. Non esiste un comportamento specifico associato a un animale generico, rafforzando la natura astratta della `class` e sottolineando l'importanza di creare **sottoclassi** specializzate per rappresentare i suoni unici di ciascun animale specifico.



Miao



Bau



Muu

```
#include <iostream>

class Animal {
public:
    // Pure virtual function to enforce implementation in derived
    classes
    virtual void speak() = 0;
};

class Cat : public Animal {
public:
    void speak() override { std::cout << "Meow!" << std::endl; }
};

class Dog : public Animal {
public:
    void speak() override { std::cout << "Bark!" << std::endl; }
};

class Cow : public Animal {
public:
    void speak() override { std::cout << "Moo!" << std::endl; }
};

void pet(Animal& animal) { animal.speak(); }

int main()
{
    // Replace `Cat` with `Dog` or `Cow` to see their specific
    behavior
    Cat cat;
```

```
pet(cat);  
}
```

Anche se non è possibile creare un **oggetto** di tipo `Animal` direttamente, è comunque possibile utilizzarlo come parametro in una funzione. Analogamente all'esempio precedente del pulsante, questo approccio sfrutta il **polimorfismo per realizzare un programma versatile che può modificare dinamicamente il proprio comportamento durante l'esecuzione**.



Nota

Prova a passare un **oggetto** di una diversa `class` alla funzione e osserva l'output. Inoltre, tenta di creare un **oggetto** della `class` astratta per vedere come il compilatore impedisce l'istanza a causa dei metodi puri virtuali non implementati.

Sovraccarico degli Operatori

L'**overloading degli operatori** è una funzionalità avanzata nei linguaggi di programmazione orientati agli oggetti che consente di **ridefinire il comportamento degli operatori per le `classes` definite dall'utente**. Sovraccaricando gli operatori, è possibile fornire implementazioni personalizzate per le operazioni che coinvolgono **oggetti** della propria `class`, rendendo il codice più intuitivo ed espressivo.

Sintassi dell'overloading degli operatori

L'overloading si ottiene **definendo funzioni membro speciali o funzioni amiche che implementano il comportamento desiderato** per l'operatore. La sintassi per sovraccaricare gli operatori varia a seconda dell'operatore che si desidera sovraccaricare. La sintassi generale è la seguente:

```
return_type operator operator_symbol(parameters);
```

La sintassi, o più precisamente, il numero di parametri e il tipo di ritorno, dipende anche dal fatto che si stiano sovraccaricando operatori come funzioni membro o come funzioni amiche.

```
class Example {  
public:  
    Example operator+ (const Example& other) {  
        // Define behavior for operator+  
    }  
};
```

```
class Example {
    friend Example operator+ (const Example& left, const Example&
right) {
        // Define behavior for operator+
    }
};
```

Tabella degli operatori che possono essere sovraccaricati

Ecco una tabella che elenca tutti gli operatori che possono essere sovraccaricati. Tuttavia, è importante notare che, sebbene sia possibile sovraccaricare questi operatori, **non è assolutamente necessario** sovraccaricarli tutti per le proprie `classes`.

Operatore	Scopo	Esempio
<code>()</code>	Chiamata di funzione	<code>function()</code>
<code>[]</code>	Indicizzazione di array	<code>array[index]</code>
<code>++</code> , <code>--</code>	Incremento/decremento	<code>++a</code> , <code>a--</code>
<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code>	Operazioni aritmetiche	<code>a + b</code> , <code>a - b</code>
<code>+=</code> , <code>-=</code> , <code>*=</code> , <code>/=</code>	Operazioni di assegnazione	<code>a += b</code> , <code>a -= b</code>
<code>==</code> , <code>!=</code> , <code><</code> , <code>></code>	Operazioni di confronto	<code>a == b</code> , <code>a < b</code>
<code><<</code> , <code>>></code>	Inserimento/estrazione di stream	<code>cout << a</code> , <code>cin >> a</code>

Sovraccarico degli operatori di inserimento stream

Per sovraccaricare gli operatori `<<` e `>>` per una `class`, si definisce tipicamente una funzione friend o una funzione membro che prende uno **stream di output** (`std::ostream&`) o uno **stream di input** (`std::istream&`) come operando sinistro e un **oggetto** della propria `class` come operando destro. Questa funzione quindi formatta i dati dell'**oggetto** e li invia verso o dal flusso.

```
class Point {
public:
    friend std::ostream& operator<<(std::ostream& out, const Point&
point);
    friend std::istream& operator>>(std::istream& in, Point& point);
private:
    int x, y;
};

std::ostream& operator<<(std::ostream& out, const Point& point) {
    return out << "x: " << point.x << ", y: " << point.y << std::endl;
}

std::istream& operator>>(std::istream& in, Point& point) {
    return in >> point.x >> point.y;
}
```

Il sovraccarico degli operatori di inserimento stream, come `<<`, consente di definire un comportamento di output personalizzato per gli **oggetti** delle proprie `classes` quando vengono inviati a uno stream di output come `std::cout`. Questa funzionalità è particolarmente utile per **migliorare la leggibilità e l'usabilità** del codice quando si lavora con tipi di dati personalizzati.



Nota

Il sovraccarico dell'operatore `<<` per il **flusso di output** è più comune rispetto al sovraccarico dell'operatore `>>` per il **flusso di input**, poiché le operazioni di input con `>>` possono essere più **soggette a errori**.

Sovraccarico degli operatori aritmetici

È possibile sovraccaricare altri operatori aritmetici (`-`, `*`, `/`, `%`) in modo simile per eseguire operazioni personalizzate con i tipi definiti dall'utente.

```
#include <iostream>

class Point {
public:
    Point(int xCoord, int yCoord) : x(xCoord), y(yCoord) {}

    Point operator+(const Point& other)
    {
        return Point(x + other.x, y + other.y);
    }

    int getX() { return x; }
    int getY() { return y; }
private:
    int x, y;
};

int main()
{
    Point p = Point(2, 4) + Point(2, 6);
    std::cout << p.getX() << ' ' << p.getY() << std::endl;
}
```

L'operatore è sovraccaricato come funzione membro della `Point` `class`. Prende un altro **oggetto** `Point` come parametro e restituisce un nuovo **oggetto** `Point` che rappresenta la somma dei due punti. L'operatore `+` può essere sostituito con `-`, `*`, `/` o `%`, con i relativi aggiustamenti alla logica di implementazione.

Incremento e Decremento

Sia gli operatori di **incremento e decremento prefisso e postfisso** (`++` e `--`) possono essere sovraccaricati per `classes` personalizzate. Iniziamo con la versione prefissa:

```
#include <iostream>

class Point {
public:
    Point(int xCoord, int yCoord) : x(xCoord), y(yCoord) {}

    // Prefix increment operator (++point)
    Point& operator++() {
        ++x; ++y;
        return *this;
    }

    int getX() { return x; }
    int getY() { return y; }
private:
    int x, y;
};

int main()
{
    Point p(2, 2);
    ++p;
    std::cout << p.getX() << ' ' << p.getY() << std::endl;
}
```



Nota

L'operatore di decremento viene sovraccaricato **in modo simile all'operatore di incremento**, utilizzando l'operatore `--` e l'operazione di sottrazione.

Come puoi vedere, il sovraccarico degli operatori di incremento e decremento **prefissi** è diretto. Tuttavia, la situazione si complica con le versioni **postfisse**. Ricorda che esiste una differenza nell'**ordine di esecuzione** tra le operazioni prefisse e postfisse.

Operatore	Sintassi	Risultato	Descrizione
++Prefisso	<code>++x</code>	6	Incrementa, poi utilizza
Postfisso++	<code>x++</code>	5	Utilizza, poi incrementa
--Prefisso	<code>--x</code>	4	Decrementa, poi utilizza
Postfisso--	<code>x--</code>	5	Utilizza, poi decrementa

Quando si sovraccaricano gli operatori di incremento e decremento postfissi, è necessario **implementare anche la caratteristica dell'ordine di esecuzione**. Apparirà in questo modo:

```
#include <iostream>

class Point {
public:
    Point(int xCoord, int yCoord) : x(xCoord), y(yCoord) {}

    // Postfix increment operator (point++)
    Point operator++(int) {
        Point temp = *this; // Creating a temp variable
        ++(this->x); // Incrementing original Point's x
        ++(this->y); // Incrementing original Point's y
        return temp; // Returning created temp variable
    }

    int getX() { return x; }
    int getY() { return y; }
private:
    int x, y;
};

int main()
{
    Point p(2, 2);
    p++;
    std::cout << p.getX() << ' ' << p.getY() << std::endl;
}
```



Nota

Il parametro intero viene passato senza nome **solo per segnalare al compilatore** che si sta sovraccaricando l'operatore di incremento postfisso. Questo è necessario perché le dichiarazioni degli operatori prefisso e postfisso sono altrimenti identiche.

Nell'implementazione dell'operatore di incremento postfisso, il valore originale dell'**oggetto** viene memorizzato in una **variabile temporanea** (**temp**). L'**oggetto** corrente viene quindi incrementato, ma l'operatore restituisce il valore memorizzato in **temp** . Questo significa che la variabile viene incrementata, ma la modifica **ha effetto solo nell'espressione successiva**, poiché l'operatore **restituisce il valore originale**.